METHODOLOGY

Converge

The Spine

Compile Intent into Autonomous Systems

Building Agentic Systems as an AI-Native Engineer

Fuzzy intent descends through gated passes — each bound to the right engine, each verified before the next. Today you stand at every gate. The dark factory replaces each gate with an eval. **Same passes. No human.**

Five universal passes · one fork · one gate

The thesis

An AI-Native engineer does not write the system. They **compile intent into it** — lowering a fuzzy requirement through a fixed sequence of passes until a machine can execute it without supervision. Each pass drops one level of altitude, binds the right engine for that altitude, and ends at a gate that must pass before the next begins.

The discipline is the gate. Today the gate is **you** — you confirm, you approve, you review. The dark factory replaces each human gate with a runnable eval. The passes do not change; only who stands at the gate. You remove yourself one gate at a time.

> **Converged means an eval passed — not that you feel done.**

Structure: five passes, one fork

Passes 1–5 are the universal spine — they run on every project, identically. After the plan is hardened, the work forks: **Plan-Driven** when the trust boundary wraps the whole system, **Task-Driven** when it wraps each unit. The harness (Pass 5) is built the same either way; only the build path differs. Both forks reconverge on one gate — *eval passes = merged.*

```
1 Intent  →  2 Structure  →  3 Decomposition  →  4 Consensus  →  5 Harness  ■  FORK
 →  eval passes = merged
```

The five universal passes

Each pass lowers altitude, binds an engine, and ends at a gate. The spine of every project.

PASS 1 Intent

Understand Requirements

ENGINE	Claude Chat / CoWork — conversational, no repo noise; interrogate the business problem.
PRODUCES	acceptance-criteria.md — what “done” means, in the customer’s terms.
GATE	**the read-back of the problem is confirmed correct**
DARK-FACTORY SWAP	human confirmation → acceptance checklist

Field echo · Spec Kit “Specify”, BMAD “Analysis”.

PASS 2 Structure

Technical Specification

ENGINE	Claude Code — holds the repo and spec in context; interrogate components against real files.
PRODUCES	architecture-contract.md — the pinned component map and boundaries.
GATE	**components & constraints are pinned; no hand-waving left**
DARK-FACTORY SWAP	human review → contract assertions

Field echo · Spec Kit “Plan”, BMAD “Solutioning”.

PASS 3 Decomposition

Features & Plans

ENGINE	Claude Code (plan / auto mode) — structural thinking over the codebase, read-only.
PRODUCES	sketch/ plans — one per system (backbone, sentinel).
GATE	**every component from Structure appears in a plan**
DARK-FACTORY SWAP	human approval → schema-valid plan + coverage check

Field echo · BMAD “epics & stories”, Spec Kit “Tasks” precursor.

PASS 4

Consensus

Brainstorm & Sharpen

ENGINE	Claude Code + Codex (adversarial) + Grill-Me-with-Docs — a different model attacks the plan; docs force ground truth.
PRODUCES	the hardened, revised plan.
GATE	**no open objection remains**
DARK-FACTORY SWAP	human consensus → adversarial review passes

Field echo · BMAD “cross-agent consensus before implementation.”

PASS 5

Harness

Control Layer

ENGINE	Skills — agents-kbs-tech-stack, caw-scaffold. You generate the harness; you do not hand-build it.
PRODUCES	agents, KBs, commands, rules, CLAUDE.md — the layer you control the build from.
GATE	**the control layer is grounded — KBs real, agents scoped, rules load**
DARK-FACTORY SWAP	human runs skills → standing infrastructure, reused across projects

Field echo · OpenAI & Fowler “Harness Engineering”; the most strongly validated pass.

Pass 5 is the one artifact that **outlives the project**. Passes 1–4 and the build are project-specific; the harness is the compounding asset you carry to the next project.

The Fork — the build

Decided at the end of Consensus (coupling is clear once the plan is sharp). Executed after Harness (which is fork-independent). The fork **is** the build: it contains both cutting the work and executing it.

The deciding question · Where does the trust boundary sit — around the whole system, or around each unit? Coupling decides.

FORK A

TOOL	SpecKit / OpenSpec / AgentSpec
TAKE IT WHEN	one tightly-coupled system — pieces only make sense together
CUT + BUILD	feed the plan to the spec framework → it generates the system as one motion
TRUST BOUNDARY	wraps the whole system
VERIFY	one end-to-end test
RISK IF WRONG	re-run the spec
THIS PROJECT	**Analytics Backbone (30%) — Dagster → dbt → DuckDB is one pipeline**

FORK B

TOOL	task-spec + engines
TAKE IT WHEN	many loosely-coupled units — each stands alone
CUT + BUILD	task-spec cuts atomic tasks (each with an eval) → engines build them: Kimi (cheap S/M), Claude (judgment), Codex (adversarial)
TRUST BOUNDARY	wraps each unit
VERIFY	per-unit runnable evals
RISK IF WRONG	re-run one task, others untouched
THIS PROJECT	**Sentinel Engine (70%) — 5 agents + tools + loop are independent units**

Both forks converge on the same gate: eval passes = merged. Fork A's eval is end-to-end; Fork B's are per-unit — the cut differs, the gate is identical. That shared gate is what makes Converge one method with two paths, and what lets either fork go dark-factory.

Grounded in current practice

Every load-bearing idea in Converge is independently confirmed by the field's leading 2026 sources. Converge is a synthesis — Spec Kit's front half, Harness Engineering's middle, Loop Engineering's endpoint — unified by one fork rule and one gate.

Converge claim	Confirmed by	Verdict
Phased descent: intent → spec → plan → tasks → build	GitHub Spec Kit, BMAD, Augment Code	✓
Gates between phases ("don't move on until X passes")	GitHub blog checkpoints; Spec Kit review points	✓
The Harness as its own Control Layer	OpenAI & Fowler Harness Engineering; Databricks	✓
KBs/rules as grounding, mechanically validated	OpenAI: linters + CI validate the KB; doc-gardening agent	✓
Adversarial cross-model consensus before building	BMAD: consensus reduces hallucination drift	✓
Eval as the trust boundary for delegation	Fowler harness feedback loop; OWASP validation gate	✓
Multi-model by cognitive task (cost / judgment / adversarial)	arXiv terminal-agent; Loop Engineering	✓
Dark factory = remove the human from the loop	Osmani Loop Engineering; OpenAI factory model	✓

The original contribution: the Fork

No established framework makes "plan-driven vs. task-driven" an explicit, coupling-based decision — each picks one lane. Converge frames the choice as a first-class decision with a rule. That is its differentiator.

The endpoint: wrap the passes in a loop

Gated passes are the human-driven form. The dark factory adds the loop — find work, assign, check, record, decide next — and runs the passes on a cadence. Automate the gates **and** wrap them in a self-feeding loop, and the human is gone.

THE INVARIANT

Every pass lowers altitude, binds an engine, ends at a gate.

You are converged when the eval passes — not when you feel done.

The Harness is the one pass that outlives the project — build once, reuse everywhere.

Remove yourself one gate at a time. What’s left is the dark factory.